

Chhand Chaughule
9594

BDA expt 9

```
✓ [1] import numpy as np  
2s import pandas as pd
```

```
✓ df = pd.read_csv('student_clustering.csv')  
0s print("The shape of data is",df.shape)  
df.head()
```

↔ The shape of data is (200, 2)

	cgpa	iq
0	5.13	88
1	5.90	113
2	8.36	93
3	8.27	97
4	5.45	110

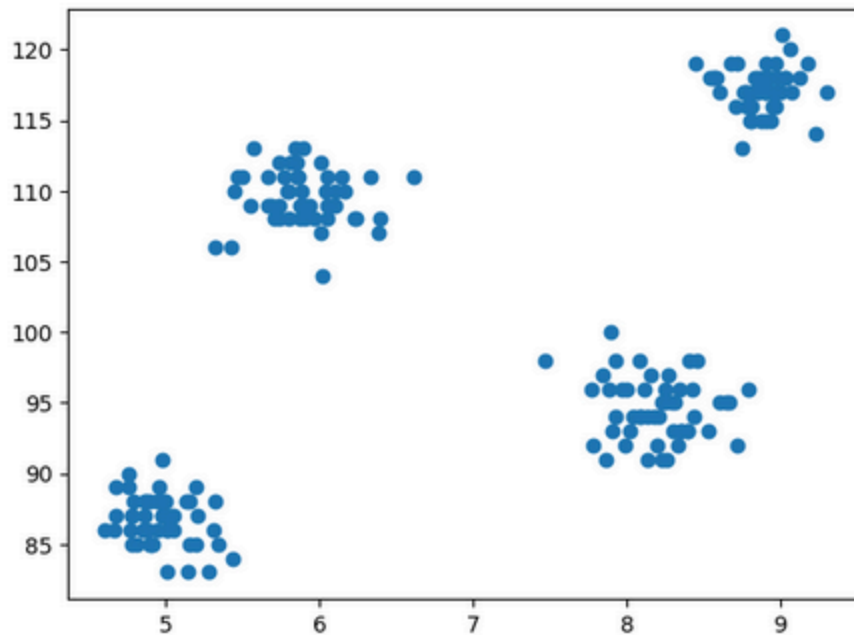
Next steps: [Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
[ ] import matplotlib.pyplot as plt
plt.scatter(df['cgpa'],df['iq'])
```

↗ <matplotlib.collections.PathCollection at 0x7acc92759030>



▶ from sklearn.cluster import KMeans

```
[ ] wcss = []

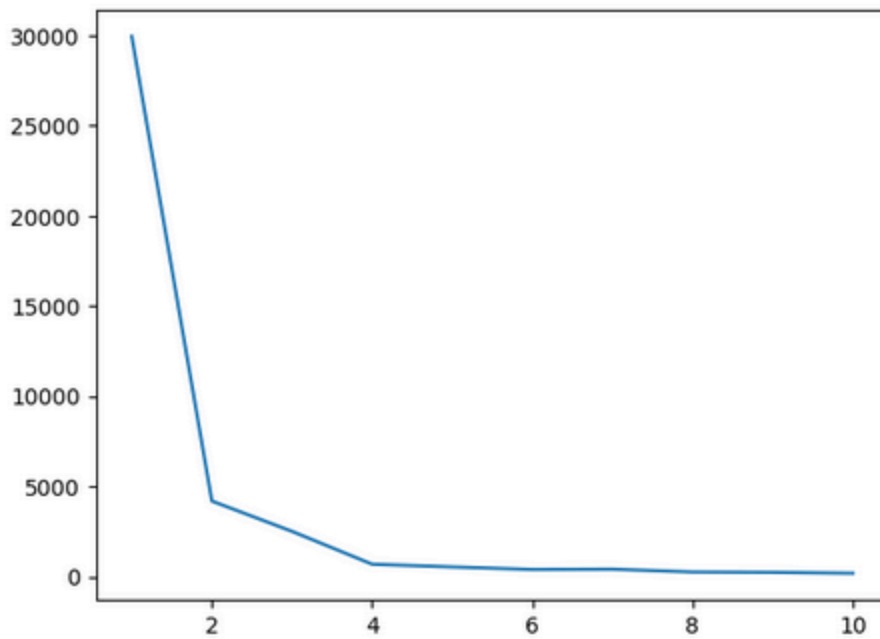
for i in range(1,11):
```

```
[ ] wcss
```

↗ [29957.898287999997,
4184.141269999999,
2503.3975810000015,
681.9696600000001,
530.4298616986187,
388.8524026875981,
405.201388356715,
250.1759664968155,
231.69866441777694,
179.83017236128757]

▶ plt.plot(range(1,11),wcss)

→ [`<matplotlib.lines.Line2D at 0x7acc8a642dd0>`]



```
✓ [8] x = df.iloc[:, :].values  
da km = KMeans(n_clusters=4)  
y_means = km.fit_predict(x)
```

```
✓ [9] y_means
```

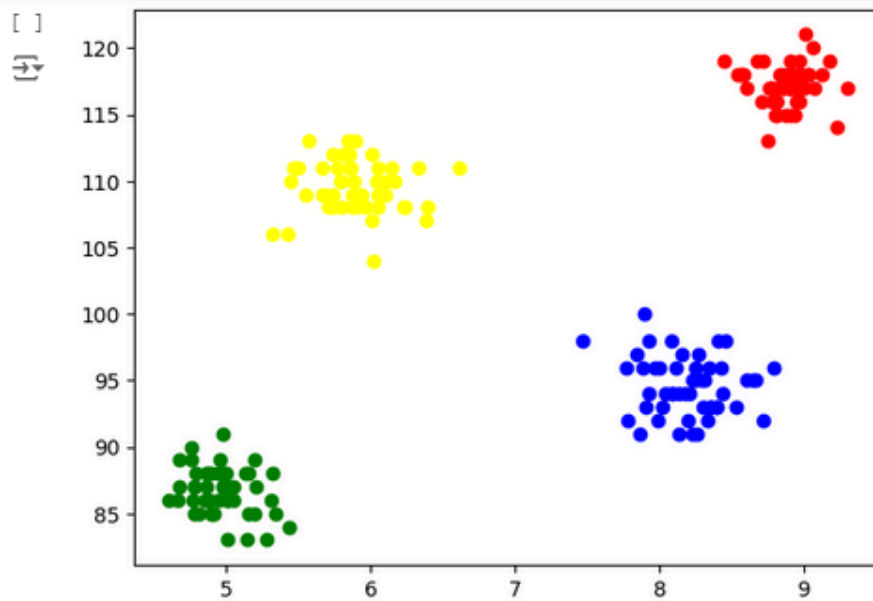
```
array([2, 3, 0, 0, 3, 3, 0, 1, 3, 0, 2, 3, 0, 2, 3, 0, 3, 0, 3, 3, 0, 2,  
       0, 2, 2, 0, 2, 1, 0, 3, 1, 3, 1, 3, 0, 0, 1, 3, 2, 3, 2, 0, 0, 2,  
       1, 1, 0, 3, 1, 3, 2, 2, 1, 0, 1, 3, 3, 1, 3, 1, 3, 0, 0, 1, 2, 1,  
       0, 2, 3, 0, 3, 1, 0, 2, 3, 1, 3, 1, 2, 0, 0, 1, 3, 2, 1, 2, 1, 3,  
       1, 3, 1, 1, 0, 2, 0, 0, 1, 0, 2, 1, 3, 2, 2, 1, 2, 2, 0, 2, 1, 1,  
       0, 1, 3, 3, 0, 1, 0, 3, 1, 2, 2, 3, 0, 1, 0, 2, 0, 3, 2, 0, 0, 3,  
       2, 2, 3, 1, 3, 2, 0, 0, 0, 2, 3, 2, 2, 1, 2, 1, 3, 2, 1, 2, 1, 1,  
       2, 0, 3, 1, 3, 0, 2, 1, 3, 0, 1, 2, 3, 2, 2, 1, 1, 3, 1, 2, 2, 0,  
       1, 3, 2, 1, 1, 3, 3, 3, 0, 2, 0, 0, 1, 3, 0, 0, 2, 2, 0, 2, 1, 3,  
       3, 1], dtype=int32)
```

```
[10] X[y_means == 3,1]
```

```
array([113., 110., 109., 110., 110., 108., 111., 111., 113., 106., 112.,  
       106., 108., 113., 108., 109., 108., 111., 109., 104., 111., 109.,  
       109., 112., 111., 109., 108., 110., 109., 111., 108., 109., 109.,  
       111., 112., 109., 111., 107., 111., 110., 108., 109., 110., 108.,  
       108., 110., 107., 111., 112., 108.])
```

```
plt.scatter(X[y_means == 0,0],X[y_means == 0,1],color='blue')  
plt.scatter(X[y_means == 1,0],X[y_means == 1,1],color='red')  
plt.scatter(X[y_means == 2,0],X[y_means == 2,1],color='green')  
plt.scatter(X[y_means == 3,0],X[y_means == 3,1],color='yellow')
```

```
<matplotlib.collections.PathCollection at 0x7acc8719e860>
```



▶ `from sklearn.datasets import make_blobs`

```
centroids = [(-5,-5,5),(5,5,-5),(3.5,-2.5,4),(-2.5,2.5,-4)]  
cluster_std = [1,1,1,1]
```

```
x,y = make_blobs(n_samples=200,cluster_std=cluster_std,centers=centroids,n_features=3,random_state=1)
```

[] x

```

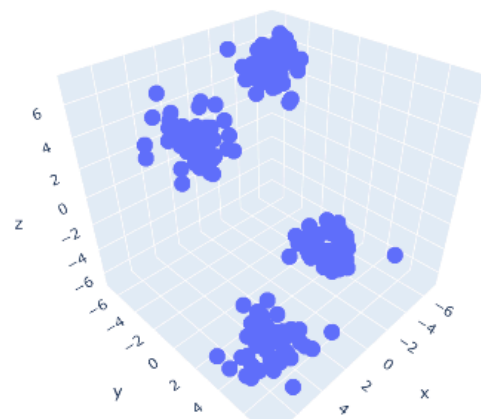
array([[ 4.33424548,  3.32580419, -4.17497018],
       [-3.32246719,  3.22171129, -4.625342  ],
       [-6.07296862, -4.13459237,  2.6984613  ],
       [ 6.90465871,  6.1110567 , -4.3409502  ],
       [-2.60839207,  2.95015551, -2.2346649  ],
       [ 5.88490881,  4.12271848, -5.86778722],
       [-4.68484061, -4.15383935,  4.14048406],
       [-1.82542929,  3.96089238, -3.4075272  ],
       [-5.34385368, -4.95640314,  4.37999916],
       [ 4.91549197,  4.70263812, -4.582698  ],
       [-3.80108212, -4.81484358,  4.62471505],
       [ 4.6735005 ,  3.65732421, -3.88561702],
       [-6.23005814, -4.4494625 ,  5.79280687],
       [-3.90232915,  2.95112294, -4.6949209  ],
       [ 3.72744124,  5.31354772, -4.49681519],
       [-3.3088472 ,  3.05743945, -3.81896126],
       [ 2.70273021, -2.21732429,  3.17390257],
       [ 4.06438286, -0.36217193,  3.214466  ],
       [ 4.69268607, -2.73794194,  5.15528789],
       [ 4.1210827 , -1.5438783 ,  3.29415949],
       [-6.61577235, -3.87858229,  5.40890054],
       [ 3.05777072, -2.17647265,  3.89000851],
       [-1.48617753,  0.27288737, -5.6993336  ],
       [-5.3224172 , -5.38405435,  6.13376944],
       [-5.26621851, -4.96738545,  3.62688268],
       [ 5.20183018,  5.66102029, -3.20784179],
       [-2.9189379 ,  2.02081508, -5.95210529],
       [ 3.30977897, -2.94873803,  3.32755196],
       [ 5.12910158,  6.6169496 , -4.49725912],
       [-2.46505641,  3.95391758, -3.33831892],
       [ 1.46279877, -4.44258918,  1.49355935],
       [ 3.87798127,  4.48290554, -5.99702683],
       [ 4.10944447,  2.8808846 , -2.0429711  ]
])

```

```

import plotly.express as px
fig = px.scatter_3d(x=X[:,0], y=X[:,1], z=X[:,2])
fig.show()

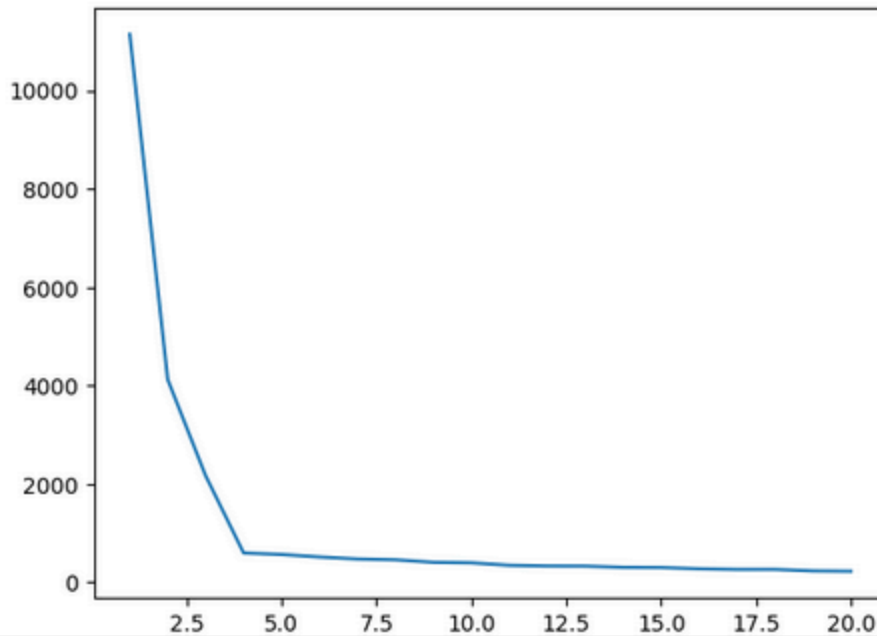
```



```
[16] wcss = []  
for i in range(1,21):  
    km = KMeans(n_clusters=i)  
    km.fit_predict(X)  
    wcss.append(km.inertia_)
```

```
plt.plot(range(1,21),wcss)
```

```
[<matplotlib.lines.Line2D at 0x7acc83c8b370>]
```

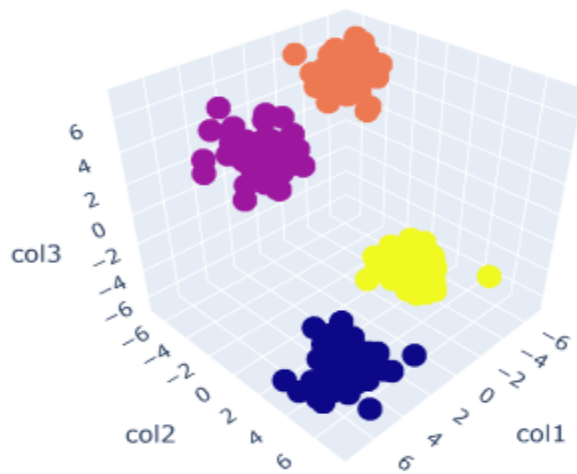


```
✓ 0s km = KMeans(n_clusters=4)  
y_pred = km.fit_predict(X)
```

```
✓ 0s [19] df = pd.DataFrame()  
  
df['col1'] = X[:,0]  
df['col2'] = X[:,1]  
df['col3'] = X[:,2]  
df['label'] = y_pred
```

```
✓ 1s [20] fig = px.scatter_3d(df,x='col1', y='col2', z='col3',color='label')  
fig.show()
```





Postlab :

K-Means clustering is one of the most widely used unsupervised machine learning algorithms for partitioning a dataset into clusters or groups. It tries to group similar data points together and discover underlying patterns in the dataset.

Steps of K-Means Algorithm:

Initialize K centroids (the number of clusters is pre-defined by the user).

Assign data points to the nearest centroid (usually using Euclidean distance).

Update centroids by calculating the mean of all data points in each cluster.

Repeat the assignment and update steps until convergence (no significant change in centroids).